

PART II — BUILDING THE INGESTION PIPELINE

Chapter 7B

Chunking Converted Documents: Repairing Structure Before Splitting

“A splitter can respect structure only after someone makes that structure trustworthy.”

Chapter 7B — Chunking Converted Documents: Repairing Structure Before Splitting

This chapter follows the real Marker-Markdown experiment from a recursive baseline through an in-memory normalization pass and full-document boundary validation. The goal is deliberately limited. We will repair enough observable structure to make splitting safer while preserving the converter output as evidence and acknowledging information that is already unrecoverable. By the end, you will be able to scope a chunking experiment, attach traceable metadata, choose where normalization belongs, test for silent loss, and know when to stop.

7B.1 The problem this chapter inherits

Definition — Structure-aware splitting

Chunking that chooses boundaries from document signals such as headings, lists, tables, and section containment rather than from character count alone.

Structure-aware splitting assumes that a heading introduces the text below it, list indentation expresses parent-child relationships, and table rows preserve their columns. Marker violated those assumptions in observed contracts and RFTs. One numbered clause appeared as a paragraph, the next as a bullet, and another as a heading. Running page banners became headings. Nested pointers flattened to one indentation level. Leading HTML anchors glued themselves to headings and prevented ordinary pattern matching.

A size-only splitter avoids trusting those signals, but it can separate a heading from its section, divide a legal clause mid-thought, or strand a list lead-in. A structure-aware splitter can do better only if the structure it reads is at least internally consistent. The experiment therefore places a small, auditable repair stage between conversion and splitting.

Analogy — Cutting a damaged film reel

A film editor uses scene markers to choose cuts. If the markers slipped during scanning, blindly cutting at each marker damages the story. The editor first realigns the markers that can be verified, then cuts, while preserving the original reel for comparison.

7B.2 Scoping the experiment

The experiment reads only `.md` and `.markdown` files below `marker_results/`. Mixing Docling, Unstructured, and Marker outputs would confound two questions: whether a splitter is good and whether each converter expresses structure differently. A fixed input family makes successive chunk reports comparable and keeps every observed defect tied to one conversion convention.

```
MARKER_RESULTS_DIR = PROJECT_ROOT / "marker_results"

def discover_files() -> list[Path]:
    return sorted(
        path
        for path in MARKER_RESULTS_DIR.rglob("*")
        if path.is_file()
        and path.suffix.lower() in {".md", ".markdown"}
    )
```

This scope is an experimental control, not a claim that Marker is the universal production parser. The expanded corpus contained legal, tender, and engineering documents whose malformed structures motivated the repair rules. A rule learned from those files must not be advertised as a general Markdown normalizer.

7B.3 Preserving traceability

Definition — Chunk provenance

Metadata that identifies the source document and the chunk's position or derivation so downstream systems can reconstruct where retrieved text came from.

Each loaded Markdown file becomes a `Document` carrying its source-relative path. After splitting, every derived chunk receives a zero-based `chunk\_seq` scoped to that source. Two files may both have chunk zero; the pair `(source, chunk\_seq)` is the meaningful identity. This is enough for inspection now and later supports neighbor-aware compression, where a retrieved chunk can request adjacent chunks from the same document.

```
sequence_by_source: dict[str, int] = defaultdict(int)

for chunk in chunks:
    source = str(chunk.metadata["source"])
    chunk.metadata["chunk_seq"] = sequence_by_source[source]
    sequence_by_source[source] += 1
```

Suppose retrieval returns chunk 17 from `marker\_results/contract.md`. The sequence metadata lets the application request chunks 16 and 18 without searching globally or assuming report order. It can inspect whether a definition began in the previous chunk or a list continues in the next. The same operation would be unsafe with a global sequence number because adjacent numbers might belong to different source files.

A production identity may add a stable source hash, converter version, preprocessing version, and chunker version. Those fields distinguish "the same path" after its contents or algorithms change. The experiment keeps the minimum pair because its immediate purpose is human inspection, but the design points naturally toward reproducible retrieval provenance.

Metadata	Scope	Why it matters
`source`	Relative file path	Links a chunk to the authoritative conversion output
`chunk_seq`	Zero-based within one source	Preserves local order and enables neighbor lookup
Character count	Per chunk	Checks the configured size bound
Token count	Per chunk and run	Estimates downstream context cost
Run timestamp	One experiment execution	Separates reports produced by different revisions

7B.4 Keeping derived artifacts out of version control

Every run writes `chunk-runs/chunks_<timestamp>.md`. The report records total chunks, average tokens, the largest chunk, and for each chunk its source, sequence, character count, token count, and full content. These files are intentionally ignored by Git because repeated experiments can produce multi-megabyte diagnostics. The code and decisions belong in history; disposable renderings of the current experiment do not.

Ignoring a report does not make it unimportant. It makes its role explicit: local evidence used to inspect boundaries, compare revisions, and locate regressions. A result promoted into a durable decision should be summarized in `Decisions.md`, `Bugs.md`, or `Research.md`, where its context survives after the large report is deleted.

Analogy — Laboratory notebooks and instrument dumps

The ledger is the signed notebook containing conclusions and measurements. Timestamped chunk reports are instrument dumps: essential during analysis, too bulky and repetitive to treat as the permanent scientific record.

7B.5 The principle — chunking is not a repair mechanism

The project adopted a necessary warning: chunking is not a repair mechanism for source-conversion defects. If Marker omitted content, swapped table rows, flattened a page frame, or discarded list nesting, a splitter cannot recover facts that no longer exist in the Markdown. Pretending otherwise turns heuristics into fabricated structure.

The warning needed qualification. Some defects do not erase information; they express the same observable structure inconsistently. A sequence such as `'5.1'`, `'5.2'`, and `'5.3'` may survive completely while appearing as a mixture of paragraphs, bullets, and headings. Normalizing those surviving markers for boundary detection is different from inventing a missing table relationship. The repair is bounded, evidence-based, and applied only to the working copy.

Common pitfall — Confusing normalization with reconstruction

A regex can make surviving clause markers consistent. It cannot infer an original nested-list depth that Marker discarded or prove which organization belonged to a corrupted table row.

7B.6 Three ways to handle malformed structure

Three implementation locations were considered. Boundary-only detection leaves text untouched and teaches the splitter additional whole-line patterns. It is the least invasive option, but it cannot rejoin a clause body fragmented by page breaks or provide consistent list syntax. Rewriting ``marker_results/*.md`` on disk makes every downstream consumer simpler, but silently mutates the very output under evaluation and destroys a clean comparison with the converter.

The chosen middle path normalizes text in memory immediately before splitting. It can repair observed clause and heading patterns while the on-disk Markdown remains authoritative evidence. The trade-off is that every downstream consumer that needs the repair must invoke the same preprocessing stage, and the heuristic must be versioned and tested like code rather than treated as cleanup magic.

Location	Advantage	Limitation	Decision
Inside boundary detection	Original text remains untouched	Cannot rejoin or normalize structure fully	Rejected as too weak
In-memory before splitting	Repairs working structure; preserves disk evidence	Consumer must invoke the stage	Chosen
Rewrite Markdown on disk	Simplest downstream representation	Destroys converter audit trail	Rejected

Repair the working copy, preserve the evidence

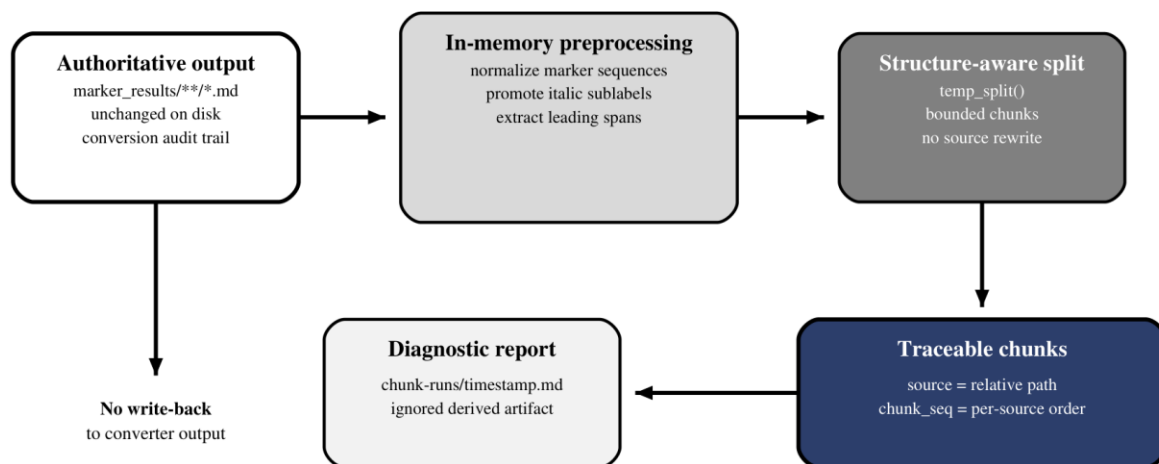


Figure 7B.1 — In-memory preprocessing repairs the working copy while preserving converter output on disk.

Figure 7B.1 shows the separation of responsibilities. ``marker_results/`` remains the evidence. ``preprocessing()`` creates a temporary normalized string, ``temp_split()`` derives chunks, and the timestamped report records what happened. There is no arrow that writes normalized content back to the converter output.

7B.7 The ``preprocessing()`` pass

The pass applies three transforms in a fixed order. ``_normalize_marker_sequences()`` detects consecutive numeric, alphabetic, or roman marker families, makes their syntax consistent, and reattaches page-fragmented body text under guarded conditions. ``_promote_italic_sublabels()`` turns a full-line italic label below a heading into the next bounded heading level. ``_extract_leading_spans()`` moves glued ``<span id="...">`` anchors onto separate lines so heading and list detectors can see the annotated line normally.

```
def preprocessing(text: str) -> str:
    text = _normalize_marker_sequences(text)
    text = _promote_italic_sublabels(text)
    text = _extract_leading_spans(text)
    return text

def split_documents(documents):
    chunks = []
    for document in documents:
        working_text = preprocessing(document.page_content)
        for text in temp_split(working_text):
            chunks.append(
                Document(text, metadata=document.metadata.copy())
            )
    return assign_sequences(chunks)
```

Worked transformation — one logical clause, three surface forms

```
# Marker output before preprocessing
5.1 Eligibility The bidder shall...

- 5.2 Submission

The complete bid shall arrive before the deadline.

### 5.3 Validity The bid remains valid...

# Normalized working text
##### 5.1 Eligibility

The bidder shall...

##### 5.2 Submission
The complete bid shall arrive before the deadline.

##### 5.3 Validity
The bid remains valid...
```

The transform does not invent clause numbers or titles; all three are visible in the input. It makes their syntax consistent and rejoins a body fragment only because the preceding clause text appears unfinished. If terminal punctuation already closes the clause, the forward merge stops. This narrow rule is why preprocessing can improve boundaries without claiming to reconstruct the original page layout.

The normalized heading level is also contextual. It may not become shallower than the nearest enclosing heading, even if one Marker fragment already carried a shallower prefix. Otherwise a child clause can parse as the sibling or parent of the section that contains it. This invariant was added only after the full document exposed the inversion.

Order matters. Marker-sequence normalization may introduce consistent headings; italic promotion then derives sublabels from the nearest preceding level; span extraction finally removes anchor prefixes that would hide the structural line. Each helper is a best-effort rule scoped to observed RFT and contract patterns. It should return the original text unchanged when no supported pattern is present.

Analogy — A transparent overlay

The normalized text is a transparent annotation sheet placed over the original document. The splitter reads the clearer overlay, but investigators can lift it away and inspect the untouched conversion underneath.

7B.8 Why headings cannot be recovered by "promote bold"

The PDF carried font size, weight, position, and spacing signals. Marker Markdown no longer contains most of that evidence. Bold is also overloaded: it marks headings, defined legal terms, labels inside clauses, warnings, and ordinary emphasis. Promoting every bold line would create a new false hierarchy and make section splitting worse.

The implemented rules therefore require stronger context: a consecutive marker sequence, a full-line italic sublabel under an existing heading, or a known leading-anchor shape. Even then, heading levels must respect their enclosing section. BUG-021 later showed why: Marker had emitted `### 5.6` beneath a `####` parent, and blindly inheriting that level made clauses siblings above their own section. The safest rule uses document context and remains conservative when evidence is ambiguous.

Common pitfall — Promoting visual emphasis after visual evidence is gone

Once font size and page position have been discarded, bold alone is not a heading classifier. Use numbering, whole-line shape, neighboring structure, and conservative scope.

7B.9 Validating chunks against full-document ground truth

Definition — Ground-truth pair

A representative input and manually verified expected output used to determine whether a transformation preserves intended structure and content.

Two toy fragments initially made the preprocessing logic look correct. A roughly 2,700-line real input/expected-output pair exposed failures those examples could not contain: unrelated marker families reused hundreds of lines apart, nested edits, glossary definitions, page anchors, and long list boundaries. Validation then expanded to the full local corpus and checked that the normalized/split output could account for every source line in rendered form.

Three invariants guide the run: no chunk exceeds `CHUNK\_SIZE`; every source line survives in some chunk after the documented normalization; and no non-final chunk ends on a heading whose content begins later. The exact round-trip comparison must allow deliberate serialization changes, such as normalized table padding or repeated headers, but it must never excuse missing semantic content.

Validation proceeds from coarse to fine. First, compare total nonblank source lines with normalized lines that appear in the emitted chunks. Next, reconstruct each source's chunks in `chunk\_seq` order and locate the first unmatched span. Finally, inspect the structural context around that offset: heading run, table, list, anchor, or paragraph. This workflow turns a million-character report into one local counterexample that can become a regression case.

A boundary is accepted for semantic reasons, not merely because the limit is satisfied. The chunk should carry the heading that labels its content, keep a list lead-in with at least its first item, and avoid cutting a table row unless the table-specific policy can repeat the applicable headers. Size is a hard constraint; coherence decides among the boundaries that remain.

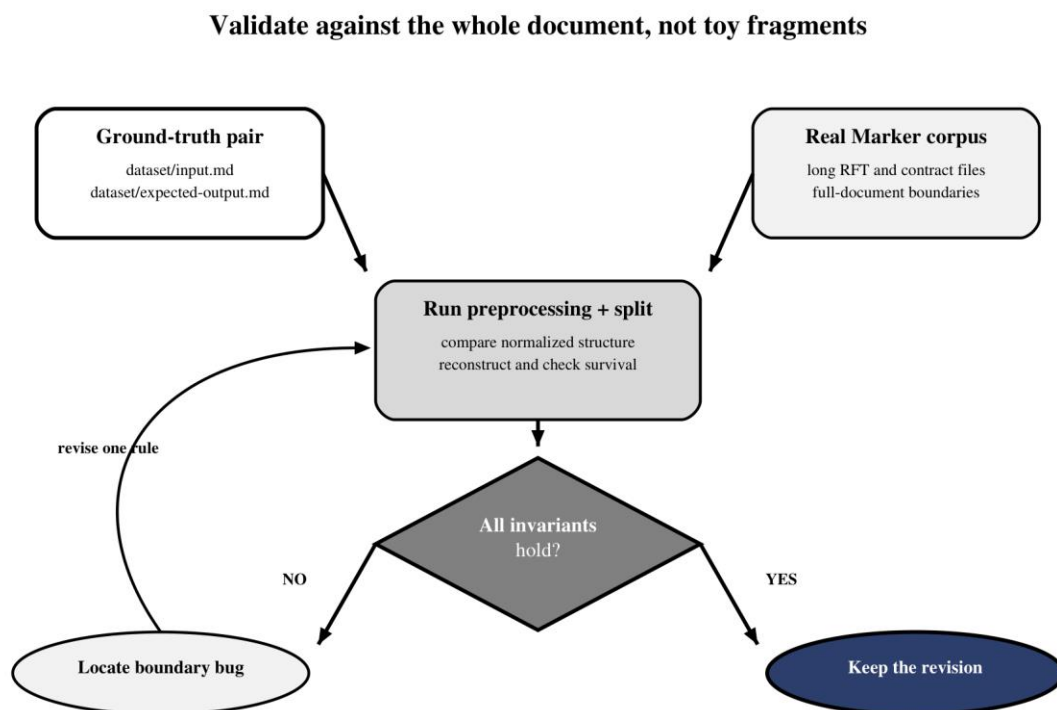


Figure 7B.2 — Full-document ground truth turns boundary defects into reproducible regression cases.

Figure 7B.2 emphasizes the loop that produced useful evidence: run the complete document, test invariants, locate the first failing boundary, revise one rule, and repeat. A passing toy example is only a unit test; a full-document comparison reveals interactions among otherwise plausible rules.

```

def assert_chunk_run(source_text: str, chunks: list[str]) -> None:
    assert all(len(chunk) <= CHUNK_SIZE for chunk in chunks)
    assert all(
        has_body_or_is_final(chunk, index, chunks)
        for index, chunk in enumerate(chunks)
    )
    assert every_normalized_source_line_survives(
        source_text, chunks
    )
  
```


7B.10 The boundary bugs

The full-document method exposed three severe failures. BUG-013 keyed alphabetic and roman sequences globally, so an `(i)` list in one section could continue a completely unrelated `(ii)` elsewhere. The fix flushes unscoped families at headings and numeric-marker boundaries and prevents overlapping nested edits from being applied twice.

BUG-014 merged every plain block after a clause until the next marker. Separate glossary definitions collapsed onto one line because the algorithm had no evidence that the clause body continued. The fix permits forward merge only while accumulated text does not appear to end a sentence. This is still heuristic, but it adds a defensible stopping signal.

BUG-015 appeared in splitting rather than preprocessing. A helper seeking top-level list boundaries omitted the current cursor when that cursor began inside a nested run. Text before the next top-level item vanished from all chunks. Inserting the actual start position restored zero nonblank gaps across the tested documents.

These bugs share a pattern: each local rule looked reasonable when considered alone. The damage appeared only when scope, stopping conditions, and cursor ownership interacted across a long document. That is why logging the final chunks is necessary but insufficient. The validator must also account for input coverage and structural ownership, otherwise a cleanly formatted report can conceal missing text.

Bug	Silent symptom	Root mistake	Guard
BUG-013	Unrelated enumerations chained	Marker family scoped globally	Flush at structural boundaries
BUG-014	Glossary blocks concatenated	Forward merge had no stop signal	Require unfinished sentence evidence
BUG-015	Nested-list content disappeared	Start cursor omitted from boundaries	Always include actual split start

Common pitfall — Testing transformations only on examples that inspired them

The original examples could not reveal document-wide marker reuse, overlapping edits, or a cursor entering mid-list. Test the rules against material large enough to contain unrelated structures.

7B.11 Bounded by the converter

Some defects remain outside the repair boundary. Marker flattened parent pointers and roman-numeral subpoints into identical top-level `` items. Once indentation and geometry are gone, a splitter cannot prove which child belonged to which parent. Similar limits apply to reordered table cells, omitted text after a cross-page transition, and page furniture that is indistinguishable from genuine body text.

The correct response is not a more aggressive regex. Preserve the limitation in metadata or quality status, quarantine high-risk documents, and improve the conversion stage. A chunker should prefer an honest flat list over a confident but invented hierarchy. Knowing where to stop is part of the algorithm.

Analogy — Restoring a torn label, not a missing page

If a label is torn but every letter remains, careful alignment can restore it. If a page is missing, rearranging the remaining pages cannot recreate its contents. Preprocessing repairs surviving signals; it does not manufacture absent evidence.

7B.12 Baseline vs custom splitter

At the decision point captured by ADR-006, the committed recursive splitter remained the baseline. It was simple, bounded at 1,600 characters with overlap, and easy to compare. A custom heading/table/list prototype attempted to preserve more structure, but repeated report inspection showed behavior that was difficult to reason about and not yet demonstrably safer. It was rejected as the replacement while its diagnostics and failure cases were retained.

That conclusion is a version boundary, not a claim that experimentation stopped. The repository later committed the prototype, replaced it with hand-authored rules, and then evolved again toward parse-then-pack in ADR-010 and bounded table serialization in ADR-011. Those later changes reinforce the lesson of this chapter: a custom splitter earns adoption only through explicit invariants, full-corpus validation, and traceable decisions. Chronology matters because each design responded to defects the previous one made visible.

Before replacing a baseline, require a fixed corpus, a frozen configuration, a reviewable report, zero unexplained content loss, no over-limit chunks, and targeted checks for headings, lists, and tables. Compare not only average tokens and chunk count but also the worst boundary. A lower chunk count can mean better packing, or it can mean unrelated sections were merged. Metrics become evidence only when paired with inspected examples.

Criterion	Recursive baseline	Custom prototype at this stage
Complexity	Small and explainable	Many interacting structural cases
Structure preservation	Limited	Promising but inconsistent
Failure visibility	Predictable size cuts	Silent boundary interactions observed
Evidence burden	Established comparison baseline	Required broader ground truth
Decision	Retain for comparison	Reject as immediate replacement

Chapter 8 now receives chunks whose provenance and limitations are explicit. It moves from textual boundaries to embeddings: how each chunk becomes a vector, why model choice and dimensionality matter, and how similarity search inherits every quality decision made during conversion and splitting.